

WFX AI ERP PLATFORM

Technical Specifications & System Documentation

PREPARED FOR:

World Fashion Exchange (WFX)

PREPARED BY:

Harshit (Roll No - 2310990876)

EMAIL:

harshit0876.be23@chitkara.edu.in

VERSION & DATE:

Version 1.0.0 | July 2026

COMPLIANCE STATUS:

100% Fully Compliant (All Mandatory & Optional Features)

1. SYSTEM ARCHITECTURE & DATA FLOWS

The platform uses a decoupled client-server architecture to separate user interaction from backend query processing, vector matching, and LLM integrations.

1.1 Core Components

- **React Frontend SPA (Vite + TS):** Provides the visual dashboard, SSE messaging chat thread, multi-modal search forms, and infinite scroll product gallery cards.
- **Express Backend Coordinator:** Exposes HTTP REST routes, handles session authentication (JWTs), parses request body parameters, and holds memory file upload buffers.
- **Supabase Database (PostgreSQL 17.6):** Stores relational tables and handles vector similarity matching via the pgvector extension.
- **OpenRouter AI Service (Llama 3.1):** Translates natural language questions to SQL SELECT statements and generates textual summary reports.
- **HuggingFace Inference (CLIP Model):** Generates 512-dimensional text and image vector embeddings to match style profiles.

1.2 Core System Data Flows

A. Natural Language Query (NL2SQL) with Chat Context:

1. Frontend captures user query and the last 3 conversation turns (6 messages).
2. Frontend opens an SSE stream via EventSource passing the query and history parameters.
3. Backend validates session token and forwards prompt metadata with schemas and context history to Llama 3.1.
4. LLM resolves pronoun references (e.g. "their prices" -> prices for styles retrieved in the previous turn) and returns SQL.
5. Backend executes SQL against Supabase using `exec_sql()` function.
6. If execution fails, backend catches database error and prompts LLM to correct the query (up to 2 retries).
7. Backend streams AI summaries token-by-token alongside raw table rows back to the frontend.

B. Multi-Modal Product Search:

1. User submits a search string or uploads a garment reference image.
 2. Backend posts input to HuggingFace CLIP to generate a 512-dimensional vector embedding.
 3. The vector is sent to Supabase to calculate similarity scores using `match_finished_goods()` function.
 4. Matches exceeding a threshold (0.80) are returned, sorted by cosine similarity confidence.
-

2. CLOUD DEPLOYMENT & REST API SPECIFICATIONS

2.1 Infrastructure Setup

- Database (Supabase): Enable pgvector extension. Execute schema.sql to create database tables and RPC functions.
- Backend Service (Render): Deploy Express server. Add variables: SUPABASE_URL, SUPABASE_KEY, JWT_SECRET, OPENROUTER_API_KEY, and HUGGINGFACE_TOKEN.
- Frontend client (Vercel): Deploy frontend directory. Set VITE_API_URL pointing to backend Render service URL.

2.2 REST API Specifications

POST /api/auth/register

Request: {"email": "user@company.com", "password": "..."}

Response: {"token": "JWT", "user": {"email": "..."} }

POST /api/auth/login

Request: {"email": "user@company.com", "password": "..."}

Response: {"token": "JWT", "user": {"email": "..."} }

GET /api/dashboard/stats

Request: None (Auth Header required)

Response: {"summary": {...}, "charts": {...}}

GET /api/products

Request: Query Parameters: page, limit, sort_by, category

Response: {"products": [...], "pagination": {...}}

GET /api/products/filters

Request: None (Auth Header required)

Response: {"categories": [...], "fabrics": {...}}

POST /api/search/text

Request: {"text": "denim jacket", "category": "Jackets", "gsmMin": 150}

Response: {"results": [...], "searchMode": "semantic"}

POST /api/search/image

Request: Multipart form-data: file in "image" key

Response: {"results": [...]}

GET /api/nlq/query

Request: Query Parameters: query, history (JSON String)

Response: Streams SSE chunk data events

3. UI REFERENCES & FUTURE IMPROVEMENTS

3.1 User Interface Screenshots

- Dashboard Screen: Displays statistics (Total revenue, invoices, active suppliers) and Recharts analytics.
[Placeholder: ./screenshots/dashboard.png]
- NL Explorer (SQL Chat): Interactive thread demonstrating SQL translation, database results, and streaming summaries.
[Placeholder: ./screenshots/nl_explorer.png]
- Product Search Filter Screen: Combined multi-modal page with semantic text search and image uploads.
[Placeholder: ./screenshots/product_search.png]
- Catalog Grid Explorer: Infinite scrolling gallery with card layouts and tech pack specification modals.
[Placeholder: ./screenshots/finished_goods.png]

3.2 Future Scaling & Performance Improvements

Tuning HNSW Indexing:

As product catalogs scale beyond 10,000 styles, adjust HNSW vector index parameters (Max Connections "m" and ef_construction) to speed up cosine similarity matches.

SQL Execution Safeguards:

Revoke database-level execute permissions on `exec_sql()` from public roles. Implement AST parsing in Express to ensure generated queries only perform read operations.

Asynchronous Embedding Generation:

Use database triggers or webhooks to process newly added styles. Offload vector calculations to background queue workers (e.g. BullMQ) to avoid blocking the main server thread.

Semantic Query Caching:

Cache frequently run natural language queries and their SQL translations in Redis. Use vector matching on incoming queries to return cached results directly, reducing LLM costs.
